# OBSIDIAN QUANT AI

Pro Master Edition • Complete Web Application Blueprint

A-to-Z architecture, UI/UX, live market data, AI routing, paper trading, testing and deployment specification

**Version:** 2.1 • **Prepared:** 7 November 2026 • **Language:** English

**Design principle:** Real provider data only; no fake live quotes, mock AI, fabricated confidence or unverified broker execution.

Important: This is a software development blueprint, not financial advice or a promise of trading profitability. Free APIs are rate-limited and may not provide exchange-grade real-time data.

# Contents

1. Project purpose and non-negotiable rules
2. Product scope and modes
3. UI/UX concept and layout
4. Architecture and flow diagrams
5. Repository structure
6. Market data providers and free-tier strategy
7. Live chart and WebSocket lifecycle
8. Binary Quant Engine
9. Forex AI Suite
10. Gemini and multi-provider AI router
11. Backtesting and market intelligence
12. Server manager and credential security
13. API and database design
14. Security, reliability and observability
15. Vercel/PWA deployment
16. QA and acceptance criteria
17. Development roadmap
18. Master coding agent prompt
19. References

# 1. Project Purpose and Non-Negotiable Rules

**App name:** Obsidian Quant AI – Pro Master Edition. Build a responsive, professional trading-analysis web application with independent Binary Options and Forex analysis engines, a historical backtest module, market intelligence, configurable market data providers, and a secure multi-provider AI analysis service.

**Critical reality check:** no single free provider reliably guarantees unlimited real-time Forex and binary-option quotes, unrestricted WebSocket connections, or production-grade execution. Provider plans, market coverage, data redistribution rights, and quotas change. The app must show limitations rather than pretending that a free endpoint is unlimited.

## Non-negotiable requirements

• No fake live prices, fake candles, fake news, fabricated AI output, or simulated connection success.
• Demo/paper mode must be clearly labeled and isolated from live brokerage execution.
• Real-money order execution disabled by default; requires an explicit supported broker adapter, user consent, secure credentials, risk checks, and broker confirmation.
• AI is advisory. It cannot autonomously place trades.
• If data is missing, stale, invalid, or quota-limited, show WAIT / DATA UNAVAILABLE and explain why.
• No guarantee of bug-free software. Build, test, fix, and verify in stages.

# 2. Product Scope and Modes

The application has six main workspaces: Binary Quant Engine, Professional Forex AI Suite, Backtest Quant Engine, Market Intel & Sentiment, Provider/API Manager, and Security/Settings.

## Operating modes

**Analysis-only:** market charts and AI analysis; no orders.
**Paper trading:** simulated orders using actual market data and clearly labeled simulated execution.
**Broker-connected:** only when an official supported broker API is configured and user has enabled it. Never imply a binary broker can be integrated without a documented API.

# 3. UI/UX Concept

Visual style: institutional terminal, clean dark SaaS, restrained neon accents. Base colors: #04060A background, #0B1018 panels, #00F3FF cyan accent, #00FF88 bullish/paper-buy, #FF4560 bearish/sell, #E8EDF5 primary text, #8B98AA secondary text. Fonts: Inter for UI and JetBrains Mono for prices/metrics.

## UI concept mockup (illustrative only; no quote is live):

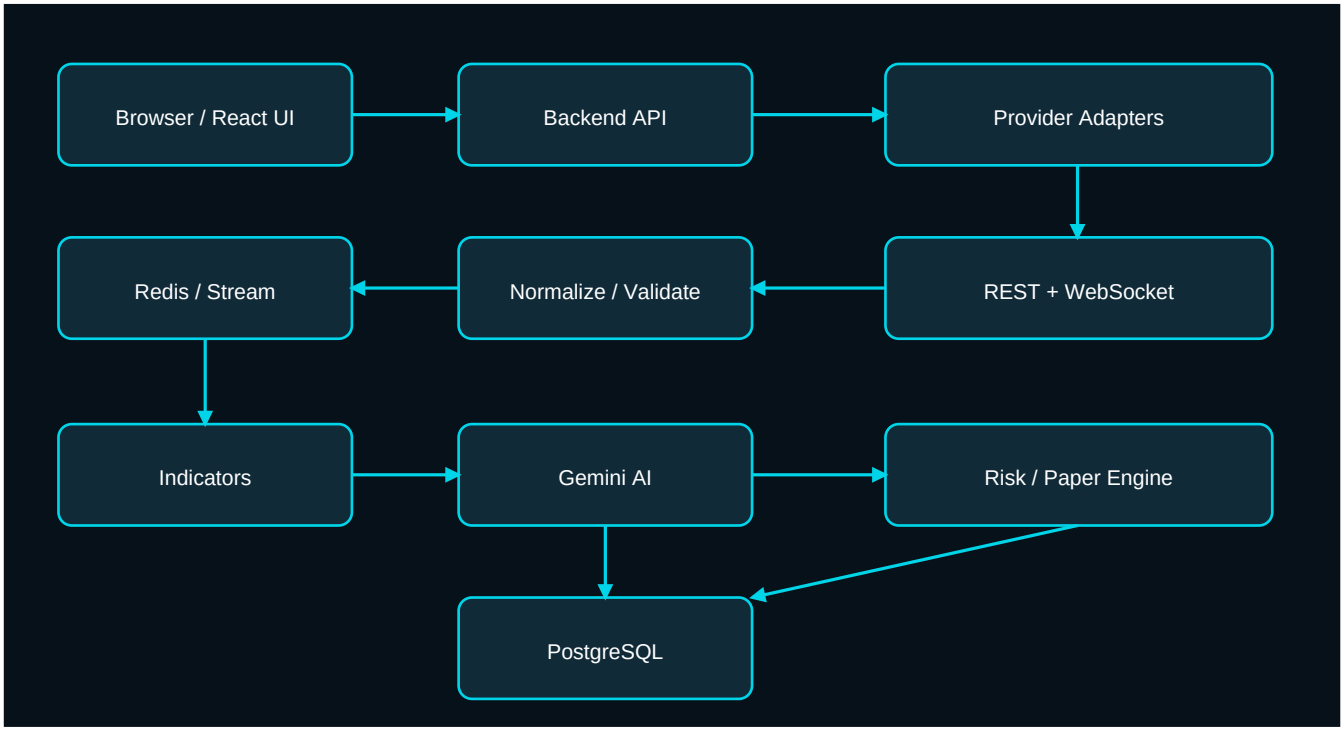



Desktop: collapsible market watch on left, chart in center, AI panel and order controls on right, open positions/history at bottom. Tablet: two-column responsive grid. Mobile: single-column, collapsible watchlist, chart controls and bottom navigation. Avoid page-wide horizontal overflow; allow scrolling only inside dense tables.

## Main navigation

Dashboard • Binary • Forex • Backtest • Market Intel • Provider Manager • Settings • Help/Status.

# 4. System Architecture




Flow: browser UI → authenticated backend → provider adapters → REST historical data and WebSocket ticks → normalization/validation → shared stream/cache → chart and indicators → AI analysis → response validation → risk checks → paper engine → database and trade journal.

## Architecture principles

• Frontend and backend are separate concerns.
• Market provider integrations live behind adapter interfaces.
• AI providers live behind a common AI adapter interface.
• PostgreSQL stores durable user, provider, signal, strategy and paper-trade records.
• Redis is optional for caching/pub-sub; app must still handle Redis outage.
• WebSocket reconnection and resynchronization are mandatory.
• Avoid coupling long-running market stream workers to short-lived serverless function memory.

# 5. Repository Structure

```
obsidian-quant-ai/
■■■ package.json
■■■ package-lock.json
■■■ vite.config.ts
■■■ tsconfig.json
■■■ index.html
■■■ tailwind.config.ts
■■■ postcss.config.js
■■■ eslint.config.js
■■■ .env.example
■■■ .gitignore
■■■ vercel.json
■■■ README.md
■■■ public/ (logo, favicon, manifest, app icons)
■■■ src/
■   ■■■ main.tsx, App.tsx, index.css
■   ■■■ app/ (router, providers, store, query client)
■   ■■■ components/
■   ■   ■■■ layout/ (AppShell, TopBar, Sidebar)
■   ■   ■■■ charts/ (CandlestickChart, ChartToolbar)
■   ■   ■■■ trading/ (MarketWatch, OrderPanel, PositionTable)
■   ■   ■■■ ai/ (SignalBrain, ConfidenceMeter, Reasoning)
```

```
■   ■   ■■■ common/ (Modal, Button, ErrorBoundary)
■   ■■■ features/ (binary, forex, backtest, intel, settings, auth)
■   ■■■ hooks/ (useMarketStream, useCandles, useSignal)
■   ■■■ services/ (apiClient, websocketClient)
■   ■■■ types/, utils/, constants/
■   ■■■ assets/
■■■ api/ (Vercel-compatible API entrypoints)
■■■ server/
■   ■■■ app.ts, config/, middleware/
■   ■■■ modules/ (market, AI, binary, forex, backtest, risk, execution)
■   ■■■ adapters/ (market providers, broker adapters, AI providers)
■   ■■■ database/, websocket/, jobs/, utils/
■■■ prisma/schema.prisma
■■■ tests/ (unit, integration, e2e, security, fixtures)
■■■ docs/ (architecture, API, security, deployment, providers)
```

Keep Vite app root compliant. If deploying persistent streaming or a worker, deploy it to a suitable long-running host or managed service; do not assume Vercel functions are immortal background processes.

# 6. Market Data Providers and Free-Tier Strategy

Use free plans for development/prototyping and personal analysis only where the provider terms permit. Free data does not imply unlimited, exchange-grade real-time data or commercial redistribution rights.

| Provider | Potential use | Cautions / engineering policy |
|---|---|---|
| Twelve Data | Forex REST candles; possible streaming depending on plan | Check current credits, symbol access, WS entitlement and terms before selecting. |
| Finnhub | Market data/news endpoints where supported | Endpoint coverage and free-tier limits vary; verify Forex feed and WS access. |
| Alpha Vantage | Historical FX and indicators / slower refresh | Request caps make it unsuitable for frequent 1-minute polling at scale. |
| OANDA practice API | Forex demo/practice account data and simulated execution | Requires account/API access; not a generic free market data source. |
| Stooq / public CSV sources | Historical end-of-day/backtest research where suitable | Not live tick feeds; confirm format, coverage and usage rights. |
| User-configured provider | Custom REST/WS provider adapter | Must pass URL, TLS, SSRF, schema and entitlement validation. |

## Provider integration interface

```
interface MarketDataProvider {
  id: string;
  name: string;
  testRest(): Promise<ConnectionResult>;
  testWebSocket(): Promise<ConnectionResult>;
  getSymbols(): Promise<MarketSymbol[]>;
  getCandles(symbol: string, timeframe: Timeframe, limit: number): Promise<Candle[]>;
  subscribe(symbol: string, onTick: (tick: MarketTick) => void): Promise<Subscription>;
}
```

Normalize every source into a common schema. Candle timestamp = UTC epoch milliseconds; OHLC must be finite and satisfy low ≤ open/close ≤ high. Missing volume must be null—not invented. Track provider, source timestamp, received timestamp, and stale status. Never mix quotes from different providers without visibly marking the source change.

## Free-tier reality and ethical quota handling

Free API quotas are provider-defined and change frequently. Do not rotate multiple API keys/accounts to evade rate limits, use unofficial scraping, or pretend quotas are unlimited. Implement a provider pool only for separately authorized user-owned keys/providers, with each provider's terms respected. On quota exhaustion, cool down and display the error; switch only to a user-approved alternative and label the new source.

# 7. Live Chart and WebSocket Engine

Use TradingView Lightweight Charts for candlestick rendering, with license attribution. Historical candles come from REST; current candle is updated from real ticks. Keep indicator calculations separate from rendering.

```
Tick → validate → deduplicate → timestamp to candle interval
     → update current candle OHLC
     → if interval changed: finalize old candle and create new candle
     → update chart series + indicators
     → store/stream snapshot as configured
```

## Tick aggregation

For a tick in the current interval: close = tick price; high = max(high, price); low = min(low, price). When a new interval starts, finalize the old candle and start a new candle. Handle out-of-order ticks explicitly; do not silently corrupt candle history.

## Connection states

CONNECTING • CONNECTED • RECONNECTING • STALE • DISCONNECTED • ERROR. Use heartbeat, timeout, exponential backoff, resubscribe, REST resync after reconnect, duplicate detection and bounded retry. If no recent ticks arrive, show last update and mark data stale.

Timeframes: Binary 1m/5m/15m; Forex 15m/1h/4h/1d. The countdown is based on exchange/provider candle boundary in UTC, displayed in user timezone. Do not imply a tick is available when market/provider is inactive.

# 8. Binary Quant Engine

UI: symbol selector; 1m/5m/15m timeframe; candlestick chart; SMA20, EMA20, RSI14, MACD, ATR, Bollinger Bands; candle close countdown; AI Signal Brain; stake selector; expiry/duration; payout; paper CALL/UP and PUT/DOWN; open paper positions; history.

## Binary payout math

Net profit on win = stake × payout rate. Total returned on win = stake + net profit. Loss on a losing trade = stake (subject to actual broker contract). Example only: $10 stake and 95% payout → $9.50 profit, $19.50 total returned on win, $10 loss. Payout must come from verified provider/broker contract; never hardcode 95% as universal.

## Binary risk controls

Paper mode default; max stake; daily loss limit; max open trades; expiration handling; no order on stale data; idempotency key; clear settlement rules; no auto execution from AI. Binary options are high risk and legal availability depends on jurisdiction and broker.

# 9. Professional Forex AI Suite

Independent Forex symbols, timeframes, chart state, indicators and position calculator. Display bid/ask/spread only if provider supplies valid values. Calculate Entry, Stop Loss, Take Profit, pip distance, position size and risk-to-reward using instrument-specific pip/contract value and account currency.

## Risk-to-reward

For a long: reward = TP – entry; risk = entry – SL; R:R = reward/risk. For short positions reverse the direction-specific distances. Validate SL/TP side, minimum tick size, spread and symbol precision.

## Forex execution

BUY/LONG and SELL/SHORT are paper orders by default. Real orders require official broker adapter, account authorization, order validation, user confirmation, broker response/receipt, and reconciliation. Do not simulate a successful broker order.

# 10. Multi-provider AI Router

Use an AI provider abstraction so user can add/configure Google AI Studio Gemini, OpenRouter, Groq, Mistral and other compatible providers. Provider models and availability must be configurable and checked against current provider docs. The system should work with one provider key at a time or user-authorized fallback providers.

## Candidate free AI providers

• Google AI Studio Gemini API: selected models have free access; quotas vary by project/model and are visible in AI Studio. Free-tier data use terms differ from paid tier.
• OpenRouter: offers free model routes, but free variants have rate limits (official docs currently list 20 RPM and 50/day for :free models; subject to change).
• Groq: free access for selected models; limits are per model/account and change; inspect official dashboard/docs.
• Mistral: free/experimental access may be available depending on account and current policy.
• Other OpenAI-compatible providers can be added through adapters, but only if the user has a valid key and the provider supports the requested model.

Never claim a provider has “high unlimited quota.” Do not rely on third-party quota comparison blogs as final authority. At setup time, show links to official pricing/rate-limit pages and require user verification. Free tier is for testing/personal use; confirm data privacy and commercial-use terms before production.

## AI Router behavior

```
1. Select configured provider/model.
2. Check local quota budget and provider health.
3. Send market snapshot to backend provider adapter.
4. Validate JSON/schema and response latency.
5. If 429/503: honor Retry-After; cooldown provider.
6. If fallback enabled: try next user-authorized provider.
7. Log provider/model/latency/error class, but redact prompts/secrets.
8. If all providers unavailable: return AI_UNAVAILABLE; do not fabricate a signal.
```

AI request should contain candle history, tick freshness, RSI/SMA/EMA/MACD/ATR, symbol, timeframe, provider/source, timestamp and market status. Keep payload compact; never send API keys or user broker secrets to model.

## AI response contract

```
{
  "symbol": "EUR/USD",
  "timeframe": "15m",
  "direction": "BUY | SELL | WAIT",
  "confidence": 0.0,
  "status": "STRONG | CONDITIONAL | WAIT | REJECTED",
  "reasoning": ["..."],
  "invalidation_conditions": ["..."],
  "risk_notes": ["..."],
  "data_timestamp": "ISO-8601",
  "model_id": "configured-model"
}
```

Confidence must be labeled “model-reported confidence” unless statistically calibrated on out-of-sample data. It is not a win probability. Invalid schema, hallucinated prices, missing data or stale market snapshot → reject / WAIT. AI is advisory and never directly executes orders.

# 11. Backtest and Market Intelligence

Backtest engine: choose instrument, timeframe, date range, strategy, indicator settings, entry/exit, SL/TP, binary expiration or Forex fees/spread/slippage. Include candle integrity checks, no look-ahead bias, no future leakage, correct timestamp alignment, out-of-sample validation and walk-forward analysis.

Metrics: trade count, win rate, net P/L, gross profit/loss, profit factor, max drawdown, expectancy, average win/loss, streaks and risk-adjusted metrics where statistically meaningful. Export CSV. Label results as historical simulation—not future guarantees.

Market Intel: provider-backed news, economic calendar, currency strength, volatility and AI sentiment. Every article/event must show source, timestamp and link. If provider fails, show NEWS UNAVAILABLE; no generated fake news. Sentiment categories: BULLISH, BEARISH, NEUTRAL, MIXED, UNKNOWN.

# 12. Provider Manager and Credential Security

Provider UI fields: name, type, REST base URL, WS URL, API key input, provider docs link, test REST, test WS, latency, last data time, plan/entitlement note, active selection, save/edit/delete.

Connection testing must actually call the endpoint and measure round-trip time. Test result means network/authentication check only—not proof of live market coverage or order permission.

SSRF protection: validate URL scheme, TLS, host allowlist/provider registry, block localhost/private IP/link-local/metadata targets, validate redirects, DNS rebinding safeguards, timeout and response size. Do not allow arbitrary custom URLs to access internal server resources. Only explicitly trusted local development can use localhost.

Secret handling: server-side secrets only; encrypted at rest; never return full secret; mask key after save; redact logs; rotate/revoke; minimum scopes; MFA for sensitive settings. API keys must not be stored in localStorage, frontend bundles, Git or .env.example.

# 13. API and Database Design

## REST endpoints

```
GET  /api/health
GET  /api/market/providers
POST /api/market/providers/test
POST /api/market/providers
POST /api/market/providers/active
GET  /api/market/symbols
GET  /api/market/candles?symbol=&timeframe=&limit=
GET  /api/market/quote?symbol=
POST /api/ai/signal
GET  /api/ai/signals
POST /api/backtest/run
GET  /api/backtest/:id
POST /api/paper/orders
GET  /api/paper/positions
GET  /api/paper/history
GET/POST /api/settings
```

Standard success envelope: success=true, data, timestamp, requestId. Standard error envelope: success=false, error.code, error.message, retryable. Validate request/response using Zod. Rate limit API routes and use idempotency keys for order creation.

## Database entities

Users, sessions, preferences, providers, encrypted credentials, provider health logs, symbols, candles (if persistence is required), AI signals, AI request logs, strategies, backtest jobs/results/trades, paper accounts/orders/positions/history, broker accounts/orders, notifications, audit logs. Use foreign keys, indexes, migrations and user isolation. Use Decimal/NUMERIC for money/prices rather than careless binary floating-point.

# 14. Security, Reliability and Monitoring

Security: authentication, authorization, MFA, rate limits, CSRF where applicable, XSS protection, SQL injection prevention, SSRF protection, secure headers, CORS allowlist, secure cookies, encrypted secrets, WebSocket auth, audit log, dependency scans and data isolation.

Reliability: error boundaries, no unhandled promise rejections, retry policy, request timeouts, graceful provider failures, duplicate order prevention, transaction-safe paper orders, backups, monitoring, alerting, and clear status UI. Provider switching must be visible to user.

Logs must not include API keys, tokens, passwords, full broker credentials, or unnecessary private financial data. Include request ID and sanitized error codes.

# 15. Deployment and PWA

Vite root structure: package.json, vite.config.ts, index.html, src/. Build output is dist/. Vercel configuration: framework Vite, build npm run build, output dist. Use Vercel-compatible API entrypoints. Persistent WebSocket stream workers may need separate hosting/managed service; do not rely on ephemeral serverless memory.

```
# .env.example (names only; never commit real secrets)
NODE_ENV=production
APP_URL=
DATABASE_URL=
REDIS_URL=
GEMINI_API_KEY=
GEMINI_MODEL=
MARKET_PROVIDER=
MARKET_API_KEY=
AUTH_SECRET=
ENCRYPTION_KEY=
BROKER_TRADING_ENABLED=false
```

PWA: manifest, icon, install prompt, service worker, update notice, cached static shell, online/offline status. Offline mode may show saved reports but must disable live signals and order execution. Do not cache private API responses without a deliberate security design.

# 16. QA and Acceptance Criteria

## Required scripts

```
npm run typecheck
npm run lint
npm run test
npm run test:integration
npm run test:e2e
npm run build
```

## Release checklist

- ■ npm install works from clean checkout
- ■ Typecheck, lint, unit, integration and E2E tests pass
- ■ production build succeeds without ENOENT
- ■ health endpoint works
- ■ real provider test works with user key
- ■ REST candle data verified against provider source
- ■ WebSocket updates, reconnect and resync tested
- ■ stale data badge works
- ■ Gemini/AI provider response validated
- ■ quota/429/503 handling tested
- ■ provider fallback only uses configured/authorized provider
- ■ no API keys in browser bundle or repository
- ■ paper order lifecycle tested
- ■ mobile/tablet/desktop layout verified
- ■ no critical console errors
- ■ README and deployment docs complete

# 17. Development Roadmap

Phase 1: scaffold, Vite, TypeScript, Tailwind, routing, theme and responsive shell.
Phase 2: backend API, authentication, PostgreSQL and provider settings.
Phase 3: market provider adapters, REST candles and live stream.
Phase 4: chart, candle aggregator, indicators and connection state.
Phase 5: Gemini/AI router, schema validation and quota handling.
Phase 6: Binary and Forex paper engines, calculators and history.
Phase 7: backtest engine and performance analytics.
Phase 8: market intel/news adapters and sentiment.
Phase 9: security, load, failure and E2E tests.
Phase 10: Vercel deployment, environment setup, production verification.

Each phase must build, test, document, and commit before moving to the next. Do not generate a huge untested code dump. Keep a TODO/known-limitations list and never mark a feature complete unless tested.

# 18. Master Coding Agent Prompt

The following prompt is also provided separately as a TXT file. It is designed to be copied into a coding agent as the single implementation directive.

# 19. Research References

Free API limits change frequently. Verify each provider's official page before implementing or deploying. Independent comparison pages are useful for discovery only, not authoritative quota guarantees.

**Google Gemini API rate limits** — https://ai.google.dev/gemini-api/docs/rate-limits
Official docs: quotas vary by model/project; RPM, TPM, RPD; limits shown in AI Studio; free tier limits can change.

**Gemini API pricing** — https://ai.google.dev/gemini-api/docs/pricing
Official docs: selected models offer free tier; free-tier data handling differs from paid tier.

**OpenRouter free models** — https://openrouter.ai/collections/free-models
Free model routing; documented 20 RPM and 50 requests/day for :free models, subject to change.

**Twelve Data docs** — https://twelvedata.com/docs
Provider API documentation; verify endpoint availability and subscription entitlement.

**Twelve Data pricing** — https://twelvedata.com/pricing
Free plan and credit/stream access subject to plan and current terms.

**Finnhub API docs** — https://finnhub.io/docs/api
Provider documentation; endpoint limits and access vary by plan.

**Alpha Vantage docs** — https://www.alphavantage.co/documentation/
Historical and FX API documentation; free tier has request caps and is not a high-frequency tick stream.

**Groq API docs** — https://console.groq.com/docs/rate-limits
Per-model limits are dynamic; check account dashboard and official rate limit docs.

**Mistral API docs** — https://docs.mistral.ai/
Free/experimental access and limits may vary by account/region.

**Vercel WebSockets** — https://vercel.com/docs/functions/websockets
WebSocket connections in functions have lifecycle/duration constraints; reconnection is needed.

**TradingView Lightweight Charts** — https://tradingview.github.io/lightweight-charts/
Chart library docs and license/attribution requirements.

**Tailwind Vite** — https://tailwindcss.com/docs/installation/using-vite
Official setup instructions.

**Gemini API key security** — https://ai.google.dev/gemini-api/docs/api-key
Keep API keys secure; use backend/server-side secrets.

End of blueprint. Use the companion TXT file for the copy-paste coding-agent prompt.